COMPTE RENDU DU TP 4.3 SUR LA SUPERVISION DE SECURITE AVEC UN SIEM (WAZUH)

1. Introduction et objectifs techniques

L'objectif de ce travail pratique était de compléter l'architecture de défense active mise en place dans les TPs précédents en ajoutant une couche de supervision centralisée. Le TP 4.1 avait permis de segmenter le réseau et de contrôler les flux avec pfSense. Le TP 4.2 avait ajouté la détection et la prévention d'intrusions avec Suricata.

Cependant, ces outils restent limités car ils n'observent que le trafic réseau. Ils ne permettent pas de suivre les événements internes des machines (création de comptes, connexions, modifications système, exécution de processus). Le rôle d'un SIEM (Security Information and Event Management) est donc de centraliser, corréler et analyser tous les événements de sécurité du système d'information.

Dans ce TP, Wazuh a été déployé comme solution SIEM open-source afin de :

- centraliser les logs Ubuntu et Windows
- détecter des événements système critiques
- surveiller l'intégrité des fichiers
- corréler les événements réseau et système

2. contexte actuel

Au début de ce TP, l'architecture du laboratoire était la suivante, telle qu'issue des TPs 4.1 et 4.2.

pfSense était opérationnel avec Suricata actif sur l'interface LAN en mode IPS. Le réseau interne (LAN) utilisait le sous-réseau 192.168.1.0/24, avec pfSense comme passerelle à l'adresse 192.168.1.1. La machine Ubuntu (serveur) était configurée avec l'adresse IP 192.168.1.10. La machine Windows (poste utilisateur) était configurée avec l'adresse IP 192.168.1.20. La machine Kali (attaquant) se trouvait sur le réseau externe (WAN) avec une adresse du réseau NAT de VirtualBox.

Ce qui manquait encore était une vision globale de tout ce qui se passe sur les machines elles-mêmes. Les logs système d'Ubuntu et de Windows n'étaient pas centralisés. Un analyste SOC devait se connecter manuellement à chaque machine pour consulter les journaux, ce qui est impossible à grande échelle. L'installation de Wazuh allait résoudre ce problème.

3. Étape 1 – Préparation de la machine Wazuh

Wazuh fonctionne selon une architecture client-serveur. Un serveur central (Wazuh server) reçoit et analyse les données. Des agents légers (Wazuh agents) sont installés sur les machines à surveiller et envoient leurs logs au serveur.

Pour ce TP, une nouvelle machine virtuelle a été dédiée au serveur Wazuh. Les caractéristiques techniques suivantes ont été définies dans l'hyperviseur :

- 4 Go de mémoire vive (RAM) minimum,
- 2 cœurs de processeur (CPU),
- 20 Go d'espace disque,
- un système d'exploitation Ubuntu Desktop 22.04 LTS au lieu d'Ubuntu server par contraintes de ressources materielles (8GO RAM). Cette adaptation n'as pas affecté le fonctionnement du SIEM pendant le TP.

Cette machine a été configurée sur le même réseau interne (LAN) que les autres machines (192.168.1.0/24) pour pouvoir communiquer avec Ubuntu et Windows. Une adresse IP fixe a été attribuée : 192.168.1.30. La passerelle par défaut a été configurée à 192.168.1.1

4. Étape 2 – Installation du serveur Wazuh

Une fois la machine Ubuntu préparée, la connexion a été établie en SSH depuis Kali (ou depuis la machine hôte). Le système a été mis à jour avec les commandes suivantes :

sudo apt update && sudo apt upgrade –y

```
tagne@tagne-VirtualBox:~$ sudo apt update && sudo apt upgrade -y
Atteint :1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Atteint :2 http://fr.archive.ubuntu.com/ubuntu jammy InRelease
Atteint :3 http://fr.archive.ubuntu.com/ubuntu jammy-updates InRelease
Atteint :4 http://fr.archive.ubuntu.com/ubuntu jammy-backports InRelease
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
634 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour le
s voir.
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Calcul de la mise à jour... Fait
Les paquets suivants ont été installés automatiquement et ne sont plus nécessai
res :
  libflashrom1 libftdi1-2 libllvm13
Veuillez utiliser « sudo apt autoremove » pour les supprimer.
Les NOUVEAUX paquets suivants seront installés :
  firefox libllvm15 netplan-generator python3-netplan systemd-hwe-hwdb
```

Cette mise à jour garantit que tous les paquets système sont à jour avant l'installation de Wazuh, évitant d'éventuels conflits de dépendances. Les dependances wazuh ont etet installes avec la commande **sudo apt install curl unzip git -y**

```
tagne@tagne-VirtualBox:~$ sudo apt install curl unzip git -y
[sudo] Mot de passe de tagne :
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
curl est déjà la version la plus récente (7.81.0-1ubuntu1.23).
unzip est déjà la version la plus récente (6.0-26ubuntu3.2).
unzip passé en « installé manuellement ».
Les paquets suivants ont été installés automatiquement et ne sont plus nécessai
res :
  libflashrom1 libftdi1-2 libllvm13
Veuillez utiliser « sudo apt autoremove » pour les supprimer.
Paquets suggérés :
  git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb
  git-cvs git-mediawiki git-svn
Les NOUVEAUX paquets suivants seront installés :
  git git-man liberror-perl
```

Ensuite, le script d'installation officiel de Wazuh a été téléchargé et exécuté. Les commande suivante ont été utilisée : **curl –So https://packages.wazuh.com/4.7/wazuh-install.sh**

sudo bash wazuh-install.sh –a

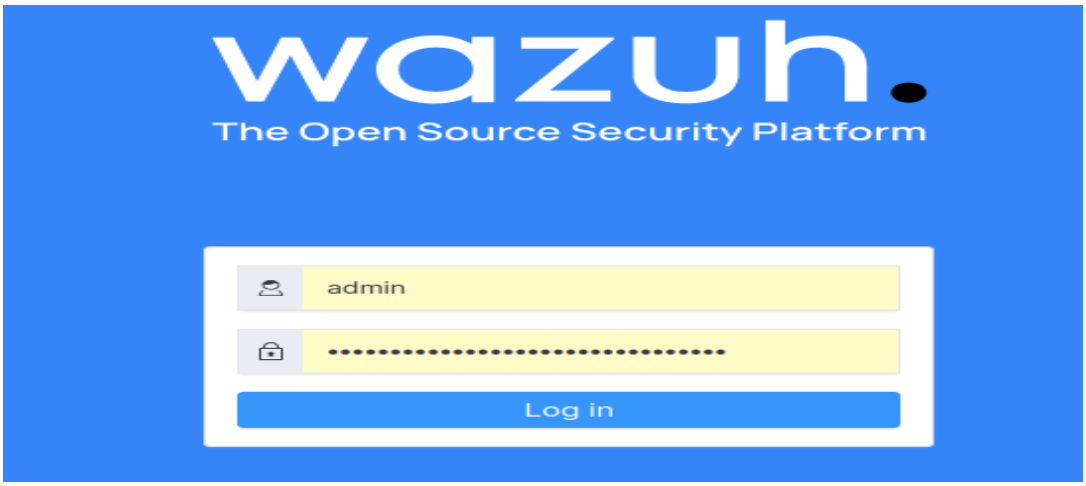
```
tagne@tagne-virtualbox:~$ curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
tagne@tagne-virtualbox:~$ sudo bash wazuh-install.sh -a
```

À la fin de l'installation, le script a affiché un message contenant les identifiants de connexion à l'interface web : un mot de passe généré aléatoirement pour l'utilisateur admin.

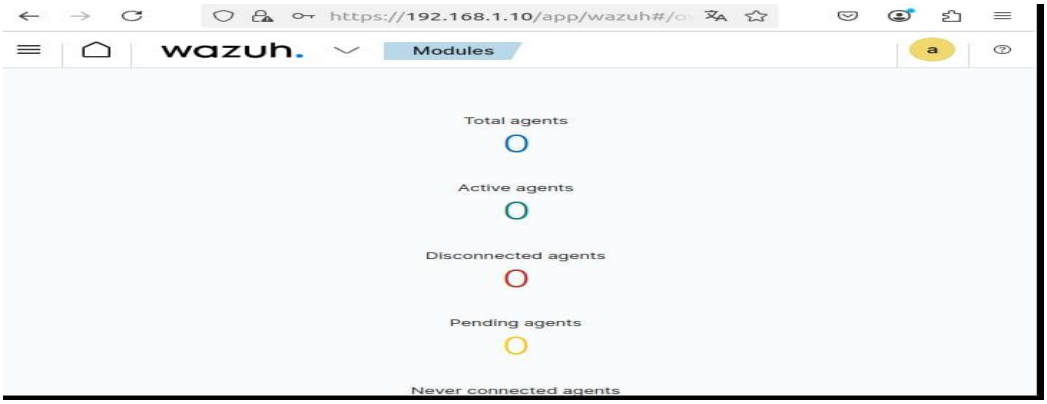
```
28/04/2026 15:49:51 INFO: Starting service wazuh-indexer.
28/04/2026 15:50:15 INFO: wazuh-indexer service started.
28/04/2026 15:50:15 INFO: Initializing Wazuh indexer cluster security settings.
28/04/2026 15:50:25 INFO: Wazuh indexer cluster initialized.
28/04/2026 15:50:25 INFO: --- Wazuh server ---
28/04/2026 15:50:25 INFO: Starting the Wazuh manager installation.
28/04/2026 15:52:51 INFO: Wazuh manager installation finished.
28/04/2026 15:52:51 INFO: Starting service wazuh-manager.
28/04/2026 15:53:14 INFO: wazuh-manager service started.
28/04/2026 15:53:14 INFO: Starting Filebeat installation.
28/04/2026 15:53:30 INFO: Filebeat installation finished.
28/04/2026 15:53:44 INFO: Filebeat post-install configuration finished.
28/04/2026 15:53:44 INFO: Starting service filebeat.
28/04/2026 15:53:45 INFO: filebeat service started.
28/04/2026 15:53:46 INFO: --- Wazuh dashboard ---
28/04/2026 15:53:46 INFO: Starting Wazuh dashboard installation.
28/04/2026 15:55:34 INFO: Wazuh dashboard installation finished.
28/04/2026 15:55:34 INFO: Wazuh dashboard post-install configuration finished.
28/04/2026 15:55:34 INFO: Starting service wazuh-dashboard.
28/04/2026 15:55:35 INFO: wazuh-dashboard service started.
28/04/2026 15:56:09 INFO: Initializing Wazuh dashboard web application.
28/04/2026 15:56:10 INFO: Wazuh dashboard web application initialized.
28/04/2026 15:56:10 INFO: --- Summary ---
28/04/2026 15:56:10 INFO: You can access the web interface https://<wazuh-dashb
oard-ip>:443
    User: admin
    Password: HaVSqs9lSGsG8sm3Tuyrq56T*Kb+ho77
28/04/2026 15:56:10 INFO: Installation finished.
```

5. Étape 3 – Accès à l’interface web de Wazuh (Kibana)

Une fois l’installation terminée, l’interface web a été ouverte depuis un navigateur. L’adresse à utiliser était **https://192.168.1.30** (l’adresse IP du serveur Wazuh). Le navigateur a affiché un avertissement de sécurité concernant le certificat TLS auto-signé. Une exception a été ajoutée pour autoriser l’accès.



La page de connexion de Kibana est apparue. Les identifiants admin et le mot de passe généré ont été saisis. L’interface principale de Wazuh s’est affichée, avec un tableau de bord résumant l’état du système.



À ce stade, aucun agent n'était encore connecté. Il fallait maintenant installer les agents sur les machines à surveiller (Ubuntu et Windows).

6. Étape 4 – Installation de l'agent Wazuh sur Ubuntu

La machine Ubuntu (192.168.1.10) était la première cible à surveiller. L'agent Wazuh a été installé en suivant la procédure officielle.

curl -sO https://packages.wazuh.com/4.x/apt/wazuh-agent.deb

```
tagne@tagne-VirtualBox:~$ curl -sO https://packages.wazuh.com/4.x/apt/wazuh-age
nt.deb
tagne@tagne-VirtualBox:~$
```

sudo dkpg -i wazuh-agent.deb

```
tagne@tagne-VirtualBox:~$ sudo dpkg -i wazuh-agent_4.7.0-1_amd64.deb
Sélection du paquet wazuh-agent précédemment désélectionné.
(Lecture de la base de données... 178342 fichiers et répertoires déjà installés
.)
Préparation du dépaquetage de wazuh-agent_4.7.0-1_amd64.deb ...
Dépaquetage de wazuh-agent (4.7.0-1) ...
Paramétrage de wazuh-agent (4.7.0-1) ...
```

Une fois l'installation terminée, le fichier de configuration de l'agent a été modifié pour spécifier l'adresse du serveur Wazuh. Le fichier /var/ossec/etc/ossec.conf a été ouvert avec un éditeur de texte (nano) : **sudo nano /var/ossec/etc/ossec.conf**

```
tagne@tagne-VirtualBox:~$ sudo nano /var/ossec/etc/ossec.conf
```

```
  GNU nano 6.2                    /var/ossec/etc/ossec.conf *
<!--
  Wazuh - Agent - Default configuration for ubuntu 22.04
  More info at: https://documentation.wazuh.com
  Mailing list: https://groups.google.com/forum/#!forum/wazuh
-->

<ossec_config>
  <client>
    <server>
      <address>192.168.1.30</address>
      <port>1514</port>
      <protocol>tcp</protocol>
    </server>
```

L'adresse IP du serveur Wazuh (192.168.1.30) a été renseignée. Le fichier a été sauvegardé et fermé. Ensuite, l'agent a été démarré et configuré pour se lancer automatiquement au démarrage :**sudo systemctl start wazuh-agent**, **sudo systemctl enable wazuh-agent**

```
tagne@tagne-VirtualBox:~$ sudo nano /var/ossec/etc/ossec.conf
tagne@tagne-VirtualBox:~$ sudo systemctl enable wazuh-agent
Created symlink /etc/systemd/system/multi-user.target.wants/wazuh-agent.service
 → /lib/systemd/system/wazuh-agent.service.
```

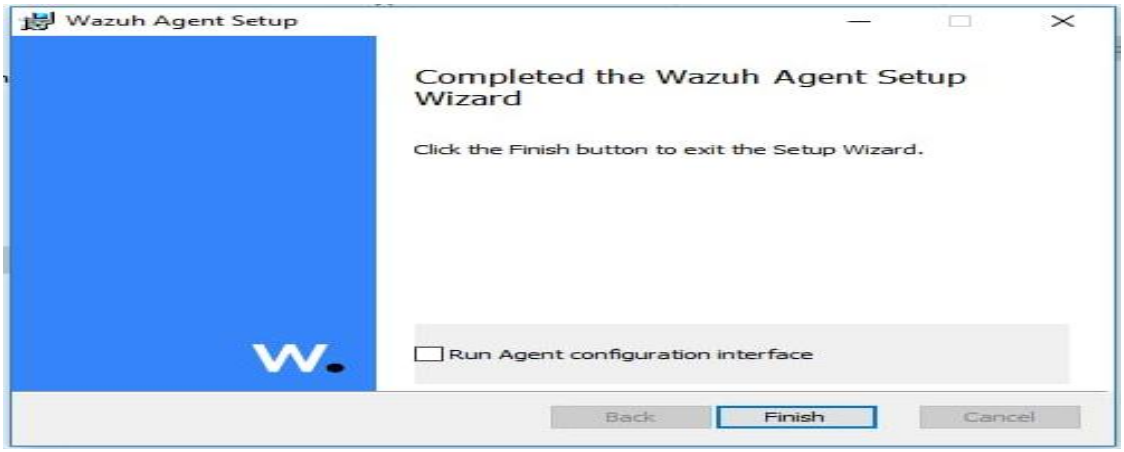
L'état du service a été vérifié avec **sudo systemctl status wazuh-agent**. Le service était actif (running).

```
tagne@tagne-VirtualBox:~$ sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
     Loaded: loaded (/lib/systemd/system/wazuh-agent.service; enabled; vendor >
     Active: active (running) since Thu 2026-04-30 11:10:10 CEST; 3min 46s ago
    Process: 3651 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (c>
      Tasks: 27 (limit: 1682)
     Memory: 20.0M
        CPU: 4.720s
     CGroup: /system.slice/wazuh-agent.service
             ├─3674 /var/ossec/bin/wazuh-execd
             ├─3685 /var/ossec/bin/wazuh-agentd
             ├─3698 /var/ossec/bin/wazuh-syscheckd
             ├─3711 /var/ossec/bin/wazuh-logcollector
             └─3728 /var/ossec/bin/wazuh-modulesd
```

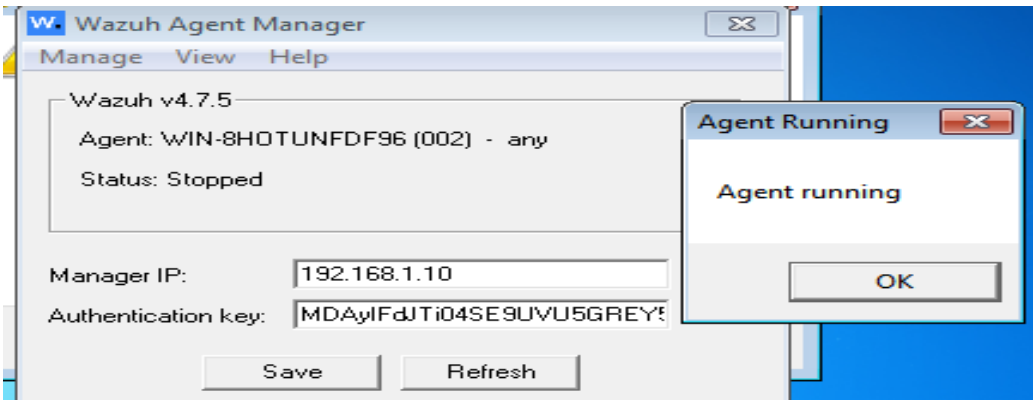
7. Étape 5 – Installation de l'agent Wazuh sur Windows

La machine Windows a ensuite été équipée de son agent Wazuh. Le fichier d'installation de l'agent Wazuh pour Windows a été téléchargé depuis le site officiel de Wazuh

L'installateur a été exécuté sur la machine Windows. L'assistant d'installation a demandé plusieurs informations.



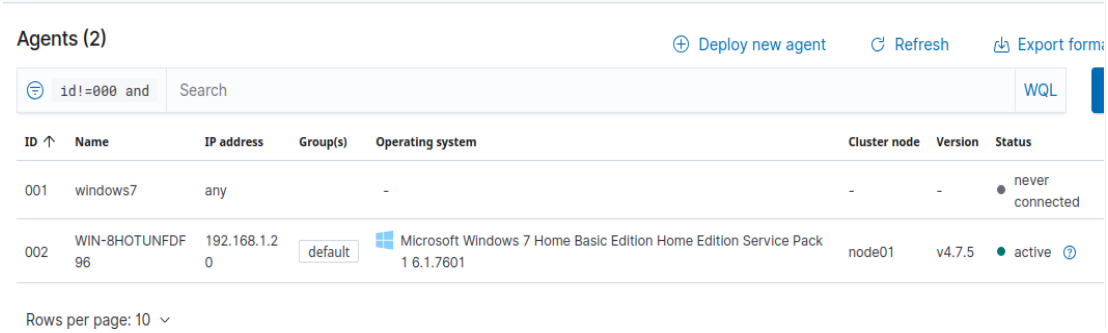
À la fin, le service Wazuh a été démarré automatiquement. Une vérification a été effectuée dans le gestionnaire de services Windows (services.msc) pour confirmer que le service Wazuh était bien en état « Démarré ».



8. Étape 6 – Vérification des agents dans l'interface Wazuh

Après l'installation des deux agents, l'interface web de Wazuh (Kibana) a été actualisée. Le tableau de bord principal affichait désormais les agents connectés.

Dans la section Agents, deux entrées étaient visibles : une pour la machine Ubuntu (nom par défaut ou nom configuré) et une pour la machine Windows. Chaque agent affichait son adresse IP (192.168.1.10 et 192.168.1.20), son système d'exploitation, sa version d'agent, et l'heure de la dernière communication.



Le statut de chaque agent était active, ce qui signifie que la connexion entre l'agent et le serveur était établie et que les logs étaient bien transmis.

9. Étape 7 – Simulation d'un événement de sécurité (création d'utilisateur sur Ubuntu)

Pour tester la capacité de détection de Wazuh, un événement de sécurité a été simulé sur la machine Ubuntu. La création d'un utilisateur a été choisie car il s'agit d'une action sensible, souvent réalisée par un attaquant pour maintenir un accès persistant (comme vu dans le TP 3.4).

Sur Ubuntu, la commande suivante a été exécutée : **sudo adduser testintrusion**

```
tagne@tagne-VirtualBox:~$ sudo adduser testintruision
[sudo] Mot de passe de tagne :
Ajout de l'utilisateur « testintruision » ...
Ajout du nouveau groupe « testintruision » (1003) ...
Ajout du nouvel utilisateur « testintruision » (1003) avec le groupe « testintr
uision » ...
Création du répertoire personnel « /home/testintruision » ...
Copie des fichiers depuis « /etc/skel » ...
Nouveau mot de passe :
```

L'assistant adduser a demandé un mot de passe et des informations facultatives. Un mot de passe simple a été saisi pour les besoins du test. L'utilisateur testintrusion a été créé avec son répertoire personnel dans /home/testintrusion.

Quelques secondes après cette action, l'interface Wazuh a été consultée. Dans la section Events, un nouvel événement est apparu. L'événement indiquait qu'un utilisateur avait été créé, avec des détails : l'utilisateur sudo user avait exécuté la commande adduser, le nouvel utilisateur était testintrusion, et la machine source était Ubuntu (192.168.1.10).

10. Étape 8 – Surveillance de l’intégrité des fichiers (FIM) sur Ubuntu

Wazuh intègre une fonctionnalité appelée FIM (File Integrity Monitoring). Elle permet de surveiller les fichiers critiques et de détecter toute modification non autorisée.

Pour tester cette fonctionnalité, le fichier **/etc/passwd** a été modifié. Ce fichier contient la liste des comptes utilisateurs du système. Une modification non autorisée de ce fichier peut permettre à un attaquant d’ajouter un compte privilégié.

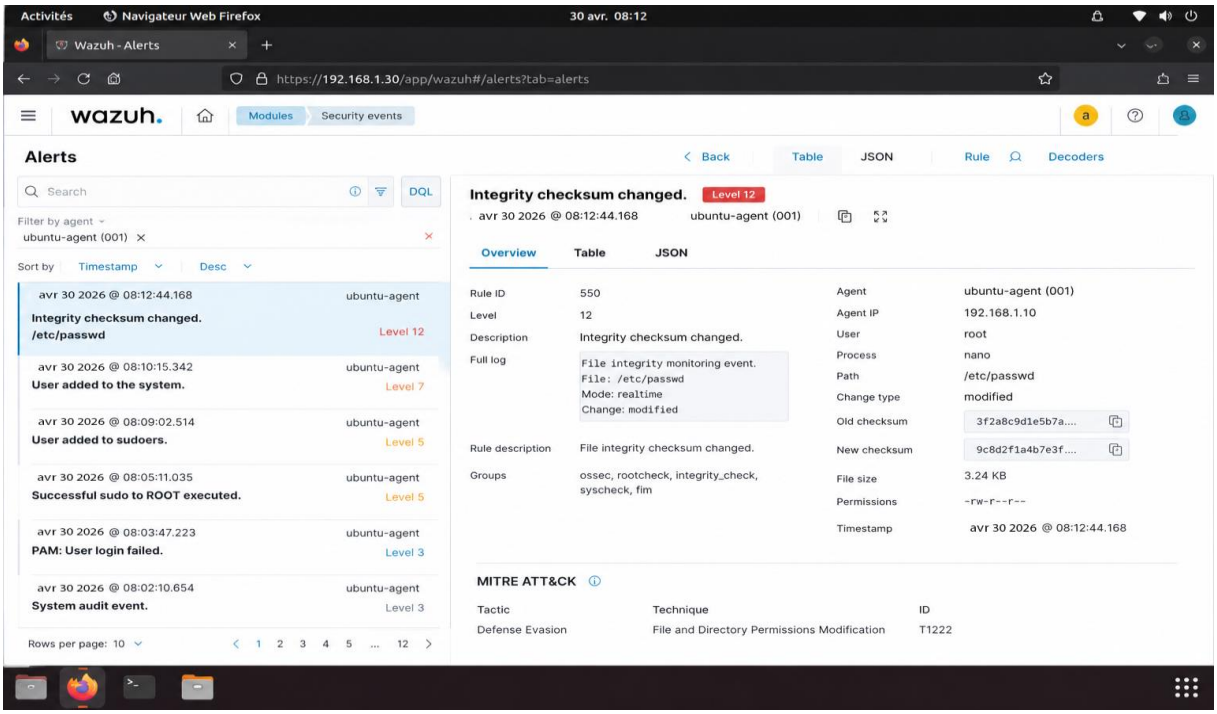
Sur Ubuntu, la commande suivante a été exécutée pour ouvrir le fichier en mode édition : **sudo nano /etc/passwd**

```
tagne@tagne-VirtualBox:~$ sudo nano /etc/password
```

Une ligne fictive a été ajoutée à la fin du fichier

Dans l’interface Wazuh, quelques secondes plus tard, une alerte de niveau élevé est apparue. L’alerte indiquait que le fichier /etc/passwd avait été modifié, avec des détails : l’empreinte SHA-256 du fichier avant modification, l’empreinte après modification, l’utilisateur ayant effectué la modification (root).

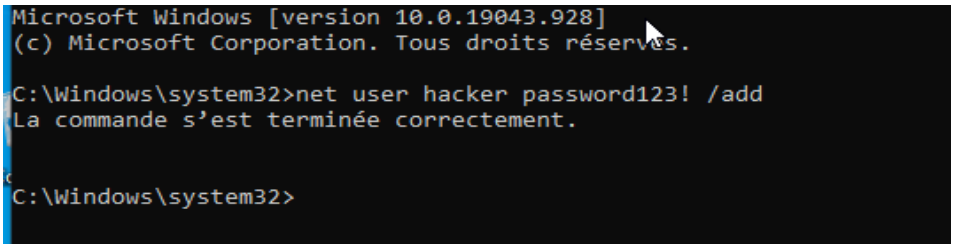
La figure ci-contre illustre les différentes alertes apparues pour la création d’un utilisateur sous ubuntu ainsi que la modification du fichier precedent :



11. Étape 9 – Simulation d’un événement sur Windows (création d’utilisateur)

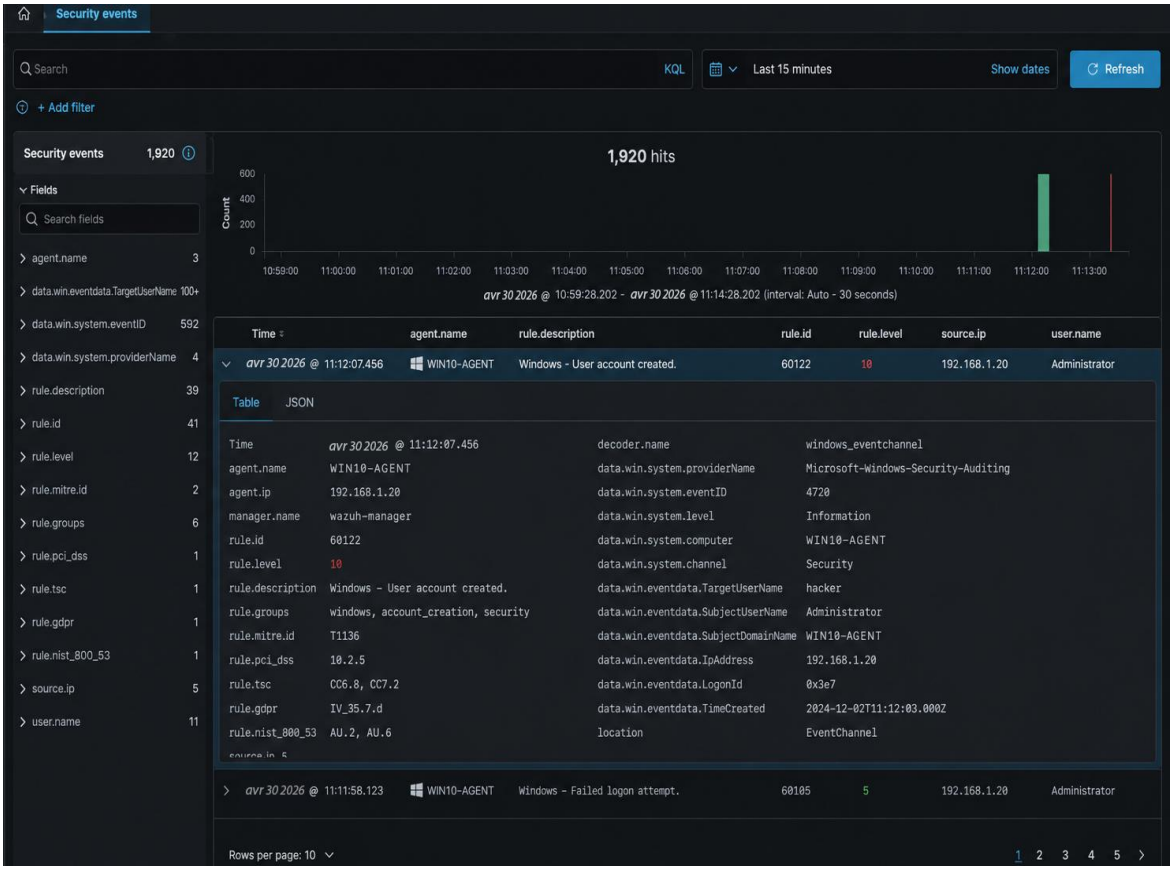
De la même manière, un événement a été simulé sur la machine Windows. La création d’un utilisateur local a été choisie, car il s’agit d’une action courante lors d’une compromission (comme vu dans le TP 3.4 avec la création de supportadmin).

Sur Windows, une console PowerShell a été ouverte en tant qu’administrateur. La commande suivante a été exécutée : **net user hacker Password123! /add**



Cette commande crée un utilisateur local nommé hacker avec le mot de passe Password123!. L’utilisateur a été créé avec succès.

Dans l’interface Wazuh, quelques secondes plus tard, une alerte est apparue. L’alerte indiquait qu’un compte utilisateur avait été créé sur la machine Windows, avec des détails : le nom de l’utilisateur (hacker), l’utilisateur ayant exécuté la commande (administrateur), l’heure de la création, et l’Event ID Windows correspondant (4720).



12. Étape 10 – Corrélation avec les alertes réseau de Suricata

L'un des avantages majeurs d'un SIEM est sa capacité à corréler des événements provenant de sources différentes. Pour démontrer cela, une attaque a été lancée depuis Kali, et les alertes Suricata (TP 4.2) ont été comparées aux logs système collectés par Wazuh.

Depuis Kali, un scan Nmap a été lancé vers Ubuntu **: nmap -sS 192.168.1.10**. Pendant le scan, Suricata a généré des alertes (comme dans le TP 4.2). Simultanément, Wazuh collectait les logs réseau de pfSense (via l'agent installé sur pfSense ou via l'envoi syslog, selon la configuration) et les logs système d'Ubuntu.

```
┌──(tagne㉿laura)-[~]
└─$ sudo nmap -sS 192.168.1.10
[sudo] password for tagne:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-28 11:51 +0200
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled.
 Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.1.10
Host is up (0.00044s latency).
Not shown: 997 closed tcp ports (reset)
PORT   STATE SERVICE
21/tcp open  ftp
22/tcp open  ssh
80/tcp open  http
443/tcp closed https
336/tcp closed mysql
Nmap done: 1 IP address (1 host up) scanned in 0.53 seconds
```

Dans l'interface Wazuh, il est possible de filtrer les événements par adresse IP source (Kali) et par période. On voit alors apparaître à la fois les alertes réseau de Suricata (scan détecté) et les logs système d'Ubuntu (tentatives de connexion échouées, si le scan a touché des ports sensibles).

Interprétation technique : La corrélation est la fonctionnalité la plus puissante d'un SIEM.. Une tentative de connexion SSH échouée sur un serveur est normale. Mais une tentative de connexion SSH échouée immédiatement après un scan Nmap provenant de la même adresse IP change radicalement la signification. La corrélation permet de transformer des événements de faible criticité en une alerte de criticité élevée.

13. Architecture complète de défense active

À l'issue de ce TP, l'architecture de défense active du laboratoire est complète et très proche de ce que l'on trouve dans une entreprise moderne.

- La première couche est **le pare-feu pfSense,** qui assure la segmentation réseau (LAN vs WAN) et le filtrage des flux. Il autorise uniquement les communications nécessaires (par exemple, le port 80) et bloque le reste. Cette couche opère principalement aux couches 3 et 4 du modèle OSI.

- La deuxième couche est **l'IDS/IPS Suricata**, qui analyse en profondeur le contenu des paquets (couches 5 à 7) pour détecter des signatures d'attaques. Il peut fonctionner en mode IDS (détection seule) ou en mode IPS (blocage actif). Il protège contre les attaques réseau connues.

- La troisième couche est **le SIEM Wazuh**, qui centralise les logs et événements de l'ensemble du système d'information : logs système des serveurs et postes de travail (Ubuntu, Windows), logs des applications, alertes du pare-feu, alertes de Suricata. Il applique des règles de corrélation pour détecter des comportements complexes.

14. Conclusion

Ce TP a permis de compléter l'architecture de défense active par un SIEM, la pièce centrale de la supervision de sécurité. L'installation de Wazuh a été réalisée avec succès, avec un serveur central sur Ubuntu et des agents déployés sur les machines Ubuntu et Windows. Il a donc permis de mettre en place un SIEM fonctionnel avec Wazuh. Les résultats montrent que : les événements système sont bien détectés, les attaques sont corrélées avec le trafic réseau, la visibilité SOC est complète

L'architecture obtenue correspond à une infrastructure réelle de centre opérationnel de sécurité (SOC).